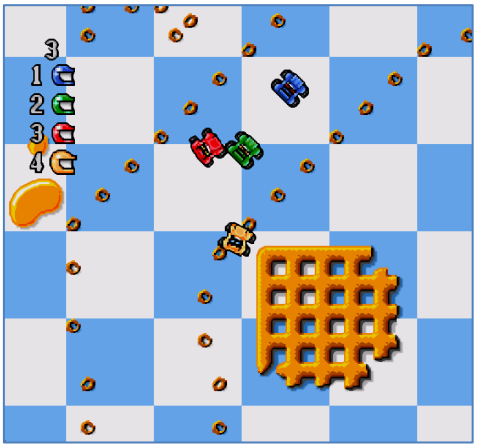
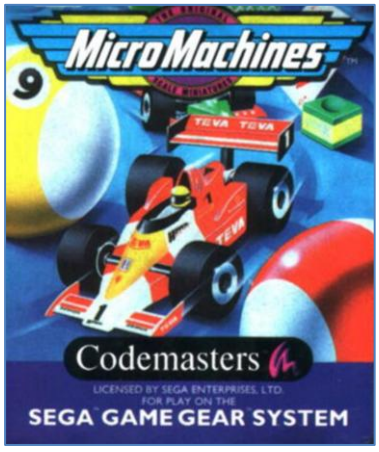
TÉCNICO LISBOA

AVTM 2026/2027

Introduction

The Micro Machines game was the first of a series based on the known toy cars sets with the same name. This game was originally released in 1991, and several versions were launched for a wide range of platforms till 2006. The player controls a car to play in unconventional environments such as a kitchen or garden. The car has to go through a track avoiding different types of obstacles. To hit or climb over obstacles the car slows down or becomes more difficult to control. You can watch about seven minutes of gameplay[1] in the channel NESguide of Youtube.





Recently several versions of the game have appeared, including some re-implementations with the original graphics, illustrated in the previous figure, with the cover of the game. Some of these versions can be played online[2].

Work to do

The main goal of AVTM Assignment 1 laboratory work this year is to recreate this classic in a 3D version, by using **C++** and **OpenGL**

[1] https://www.youtube.com/watch?v=BMpZznee74I
[2] http://www.classicgamesarcade.com/game/21618/micro-machines-driving-racing-game.html

version 3.3. The idea is to maintain the original gameplay but changing the graphic perspective for a 3D appearance. You can see an example for inspiration in the figure below.



The laboratory work corresponding to the Micro Machines 3D-based graphic app is divided into 6 lab classes where each Group will perform the resolution of several exercises and 1 assignment. The assignment 1 will be evaluated according to a calendar provided in the Presentation class and corresponds to a certain percentage of the final grade. In each of these assessments there are specific objectives and tasks so they can explore the various components of the AVTM program.

The rest of this document refers to the tasks for Exercise 2 to be developed in the 2 lab classes of week 2 (September 16th and September 18th).

Exercise 2 (week 2, September 16th and September 18th)

Objectives

The objectives for the two lab classes of week 2 are to understand and implement the architecture of an interactive graphical application and explore the basics of modeling, geometric transformations and lighting.

Tasks

The tasks for this Exercise are:

1. Model the flat table as well as the road and their margins by using a cube for each element. Model the car, orange and small packets of butter, by using simple three-dimensional geometric objects (cubes, cylinders, cones, spheres and torus). The car should be composed of more than a geometrical object, a minimum of five objects. It is suggested to represent the car's wheels and cheerios with torus. For now, you can use the **basic_geometry.cpp** library available in the VS 2026 myAVTM_Template project as discussed in Exercise 1. Later, Groups can improve the graphic quality of the models by using modeling tools and the Assimp tool.

2. Set three virtual cameras: a fixed camera (satellite camera) to provide a top view of the scene by using an orthogonal projection (like a 2D view of the original game), another fixed but perspective camera to provide also a top view, and a third moving perspective camera, which must be placed behind the car and follow your movement (the car should be visible). It must be possible to switch between the three cameras by using the keyboard keys “1”, “2” e “3”.

3. To orient the moving camera with the mouse movement and simultaneously pressing its left key.

4. To control the movement of the car with the keyboard using the 'A' key to move the vehicle to the left, 'D' to move to the right, 'W' to move forward and 'S' to move backward. The car should not reach full speed immediately after pressing the key or stop immediately when the key is released. The car should have a uniformly accelerated movement, considering as scalar the velocity and acceleration: the direction of movement is defined by a 3D vector.

5. To start implementing the movement of the oranges. This should be a uniform rectilinear movement, but these elements should rotate as they are moving. Different oranges should move with different speeds and their speed increase with the playing time, i.e., as the user is playing longer, faster oranges move. After

leaving the field of play (table) oranges should disappear and reappear **randomly** after a while elsewhere in the table.

6. Program the shaders to perform the lighting and shading of the scene using the concepts of materials, directional light sources, point lights, spotlight, Phong-Blinn reflection model and Phong shading. Thus:

   - Set the suitable materials for the car, butters, oranges, the table, and the road.
   - Create the global illumination of the scene by using a directional light source. This light source must be turned on or off via a button ('N' key) that alternates between day and night mode.
   - Create a total of six candles (Point Lights) distributed on top of the table to partially illuminate it. This illumination must be sufficient to be able to drive, but do not need to cover the entire gaming area. These light sources should be activated or deactivated via a button ('C' key) that turns on and off all six lamps.
   - Create two spotlights for the car headlights. These spotlights indicate where the car is facing and should be turned on or off through the 'H' key. Check the spotlight effect implemented in the myAVTM_Template.

7. To detect the car's collision with oranges, butters and road limits (the cheerios) by using surrounding boxes aligned with the axis (axis aligned bounding boxes - AABB). When the car reaches the limits of the track (collision with Cheerios) or a butter, it should stop, even if the user continues to press the button. When collided by the car, the butters or cheerios move slightly. If the car is hit by an orange, the player loses the race, and the car should be placed at the start line.

8. Apply simultaneously at least two textures to the table. These textures should react to the lighting. In the template, we are already loading 3 textures. See the image maps files (stone.tga, checker.png and lightwood.tga) in the Project directory. We use the variable texMode to decide how to apply them to each object. Check the different modes available and how they blend the

texture and shading information in the fragment shader. Choose the appropriate mode and image file for your table.